

Adversarially-Robust Inverse Mechanism Design (ARIMD)

A Stackelberg framework for env-design autoresearch

1 Setting

We consider a multi-agent sequential social dilemma (SSD) G with agent set $[N] = \{1, \dots, N\}$, episode horizon T , and per-agent stage rewards $r_t^{(i)} \in \mathbb{R}$. The cumulative return of agent i is $R_i = \sum_{t=1}^T r_t^{(i)}$.

We fix:

- (i) A bounded *edit grammar* $\mathcal{E}_G$ over the environment’s parameters and rule toggles. An element $e \in \mathcal{E}_G$ instantiates a concrete env $G(e)$ that differs from the default G only along the grammar’s listed axes.
- (ii) A frozen *cooperator policy* π_B^{coop} — a hand-crafted reference behaviour, the same across runs.
- (iii) An adversary fraction $\lambda \in [0, 1]$. In every episode the agents are partitioned into a Red subset $\mathcal{R} \subseteq [N]$ with $|\mathcal{R}| = \lfloor \lambda N \rfloor$ (re-sampled per seed) and a Blue complement $\mathcal{B} = [N] \setminus \mathcal{R}$. Red agents play a policy π_R chosen by the adversary; Blue agents play π_B^{coop} .
- (iv) A *welfare functional* Φ that maps the per-agent return vector $\mathbf{R} = (R_1, \dots, R_N)$ to a scalar. Default: Blue-welfare, $\Phi(\mathbf{R}; \mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} R_i$. Population-welfare, $\frac{1}{N} \sum_i R_i$, is reported alongside.

The mixed-population value of an env-policy pair is

$$V_\lambda(e, \pi_R) = \mathbb{E}_{S, \mathcal{R}} [\Phi(\mathbf{R}; \mathcal{B}) \mid G(e), S, \pi_R, \pi_B^{\text{coop}}], \quad (1)$$

where the expectation is over episode seeds S and the random Red-index assignment $\mathcal{R}$. In practice we Monte-Carlo this with a held-out seed grid.

Red is rational and *selfish*: it optimizes its own per-capita return,

$$\Phi_R(\mathbf{R}; \mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} R_i. \quad (2)$$

2 The Stackelberg meta-game

Blue commits first to an env design; Red best-responds with the most exploitative homogeneous-Red policy it can find. The *adversarially-robust mechanism* is the solution of

$$e^* = \arg \max_{e \in \mathcal{E}_G} \min_{\pi_R \in \Pi_R} V_\lambda(e, \pi_R), \quad (3)$$

where Π_R is the space of well-formed exploit policies (deterministic functions of the env state returning legal actions; in our implementation, Python policies sampled from a language model and validated by an AST-level safety check).

Algorithm 1 ARIMD-Stackelberg outer loop

Require: env grammar $\mathcal{E}_G$, cooperator π_B^{coop} , adversary fraction λ , eval seeds S , rounds T , Red candidates per round M

Require: LLM oracles $\mathcal{D}$ (Blue) and $\mathcal{X}$ (Red)

```
1:  $e \leftarrow e_0$  ▷ identity patch (default env)
2:  $v^* \leftarrow V_\lambda(e, \pi_B^{\text{coop}})$  ▷  $V_0$  self-play baseline
3:  $\mathcal{H} \leftarrow \emptyset$  ▷ round history shown to  $\mathcal{D}, \mathcal{X}$ 
4: for  $t = 1, \dots, T$  do
    // Red search at frozen Blue env  $e$ 
5:   for  $k = 1, \dots, M$  do
6:      $\pi_R^{(k)} \leftarrow \mathcal{X}(G(e), \pi_B^{\text{coop}}, \mathcal{H})$ 
7:      $v_R^{(k)}, v_B^{(k)} \leftarrow \text{EVALMIXED}(e, \pi_R^{(k)}, \lambda, S)$ 
8:   end for
9:    $k^* \leftarrow \arg \max_k v_R^{(k)}$  ▷ selfish-defector best-response, eq. (2)
10:   $\pi_R^* \leftarrow \pi_R^{(k^*)}$ 
    // Blue patch proposal
11:   $\delta \leftarrow \mathcal{D}(G(e), \pi_B^{\text{coop}}, \pi_R^*, \mathcal{H}, \mathcal{E}_G)$ 
12:   $e' \leftarrow \text{APPLYPATCH}(\mathcal{E}_G, e, \delta)$ 
    // Re-evaluate against the same Red set
13:   $v'_B \leftarrow \min_k V_\lambda(e', \pi_R^{(k)})$ , estimated over  $S$ 
14:  if  $v'_B > v^*$  then
15:     $e \leftarrow e'; v^* \leftarrow v'_B$  ▷ accept
16:  end if
17:   $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\delta, \pi_R^*, v'_B, v^*)\}$ 
18: end for
19: return  $e^* \leftarrow e$ , history  $\mathcal{H}$ 
```

Compared to the symmetric pipeline-minimax $\max_{c_B} \min_{c_R} V_\lambda(c_B, c_R)$ in which both sides choose pipeline configurations, formulation (3) gives Blue and Red *different action spaces*: Blue’s lives in a small combinatorial–continuous grammar, Red’s lives in an open program space. This (a) eliminates inner-loop policy synthesis on Blue’s side, (b) reduces LLM-collusion risk between the two proposers, and (c) yields an interpretable headline artifact (the $\mathcal{E}_G$ -diff at e^*).

Diagnostics. For an accepted env e^* and the Red best-response π_R^* we report three quantities:

- **Invasion barrier** $\lambda^* = \sup\{\lambda : V_\lambda(e^*, \pi_R^*) \geq V_0(e^*, \pi_B^{\text{coop}})\}$ — the largest adversary fraction the mechanism tolerates without falling below the no-attack baseline.
- **Exploitability** $\Delta_{\text{ex}}(J') = V_\lambda(e^*, \pi_R^*) - V_\lambda(e^*, \tilde{\pi}_R)$, where $\tilde{\pi}_R$ is obtained from a fresh Red search of J' extra iterations against frozen e^* . $\Delta_{\text{ex}} \approx 0$ certifies near-equilibrium.
- **Stability gap** $|V_\lambda(e_t, \pi_{R,t}^*) - V_\lambda(e_{t-1}, \pi_{R,t-1}^*)|$ across accepted Stackelberg rounds; monotone decrease evidences convergence.

3 Algorithm

ARIMD instantiates (3) as a finite outer loop with two LLM oracles: a *designer* $\mathcal{D}$ (Blue) and an *exploiter* $\mathcal{X}$ (Red). $\mathcal{D}$ emits a JSON patch δ over $\mathcal{E}_G$; $\mathcal{X}$ emits a Python policy. Both are validated and clamped before evaluation. Acceptance is greedy: a Blue proposal is kept only if it improves the worst-case Red value over *all* Red candidates produced in the round.

The mixed-population evaluator. $\text{EVALMIXED}(e, \pi_R, \lambda, S)$ runs $|S|$ episodes on $G(e)$. Each episode draws a fresh Red index set $\mathcal{R}$ with $|\mathcal{R}| = \lfloor \lambda N \rfloor$ uniformly at random; agents in $\mathcal{R}$ act under π_R and the rest under π_B^{coop} . We report $v_R = \frac{1}{|S|} \sum_S \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} R_i$ (Red’s objective) and $v_B = \frac{1}{|S|} \sum_S \frac{1}{|B|} \sum_{i \in B} R_i$ (Blue’s welfare). $\lambda = 0$ is a special case: π_R is ignored and the run measures pure Blue self-play.

Cost model. For T outer rounds, M Red candidates per round, $|S|$ seeds, and one episode per seed, Algorithm 1 runs $T(M + M)|S|$ episodes plus T Blue LLM calls and TM Red LLM calls — $\mathcal{O}(TM|S|)$ in episodes, linear in LLM calls. No inner policy-synthesis loop appears on Blue’s side, which is the principal source of the cost reduction over symmetric pipeline-minimax variants.

Why a bounded grammar. $\mathcal{E}_G$ exposes a small, semantically meaningful action space to $\mathcal{D}$: numeric knobs (with absolute bounds and type-preserving clamping) and structural toggles (booleans / small categoricals). This is what makes the headline artifact interpretable — e^* is a short JSON patch — and what distinguishes ARIMD’s contribution from black-box parameter search. If, in any given run, $\mathcal{D}$ ’s edits cluster on saturated numeric bounds and ignore the structural toggles, that is itself reportable evidence that the grammar (or the mechanism-design problem on this env) does not require qualitatively novel rules to defend against the chosen Π_R .